UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ingeniería

BASES DE DATOS

TAREA 8

Bojorquez Covarrubias Evans Martin

2026-2

Professor:
Fernando Arreola Franco
Grupo 1

12 de abril de 2026

1. Investigar Tipos de datos en PostgreSQL

1.1. Tipos Numéricos (Enteros y Púnto Flotante)

PostgreSQL ofrece varios tipos de datos numéricos. Los más comunes se dividen en enteros y números de punto flotante (decimales inexactos, ideales para cálculos científicos donde la velocidad importa más que la precisión absoluta al centavo).

- **Capacidad de almacenamiento:**
  - smallint: 2 bytes (Rango: -32,768 a +32,767).
  - integer (o int): 4 bytes (Rango: -2,147,483,648 a +2,147,483,647).
  - bigint: 8 bytes (Rango: hasta +-9 trillones).
  - real: 4 bytes (Precision variable: $\tilde{6}$ dígitos decimales).
  - double precision: 8 bytes (Precisión variable, $\tilde{1}5$ dígitos decimales).
  - numeric: Números hasta con 131,072 dígitos antes del punto decimal y hasta 16,383 dígitos después del punto decimal.
    - **Forma de representación del numeric:** Se declara como numeric(precisión, escala), donde la precisión es el número total de dígitos y la escala son los decimales).
- **Formato de representación:** Números estándar sin comillas (ej. 42, -15.5, 1.2e+4).

1.1.1. Ejemplos de casos de uso:

- smallint: Edad de una persona, número de puestas de un coche. (Ej. 4)
- integer: Indentificadores de usuarios (ID), cantidad de likes en un post normal. (Ej. 15420)
- doble precision: Coordenadas geográficas, mediciones científicas de peso o distancia. (Ej. 20.43568)
- numeric(10,2): Transacciones financieras, precios, salarios, tasas de interés, porcentajes exactos.

1.2. Tipos de Caráctes (Texto)

Se utilizan para almacenar cadenas de texto. PostgreSQL tiene tres tipos principales.

- **Capacidad de almacenamiento:**
  - char(n): Longitud fija. Rellena con espacios en blanco si el texto es más corto que n. Almacena hasta 1 GB.
  - varchar(n): Longitud variable con un límite máximo de n. Almacena hasta 1 GB.
  - text: Longitud variable ilimitada (hasta el máximo de 1 GB). Es el más recomendado por PostgreSQL si no necesitas forzar un límite específico.
- **Formato de representación:** Cadenas encerradas en comillas simples ('texto').

1.2.1. Ejemplos de casos de uso:

- char(2): Códigos de países estándar (ISO 3166). (Ej. 'MX', 'ES', 'US'). Siempre serán dos letras.
- varchar(50): Nombres de usuario o correos electrónicos cortos donde quieres poner un límite en la interfaz. (Ej. 'evansbojorquez@gmail.com').
- El cuerpo de un artículo de blog, descripciones de productos, comentarios. (Ej. 'Este es un articulo sobre...').

1.3. Fecha y Hora (Date / Time)

Para almacenar información temporal.

- **Capacidad de almacenamiento:**
  - date: 4 bytes (Solo fecha, desde el 4713 a.C. hasta el 5,874,876 d.C).
  - time: 8 bytes (Solo hora, sin zona horaria).
  - timestamp: 8 bytes (Fecha y hora juntas).
  - timestamptz: 8 bytes. Timestamp con zona horaria. PostgreSQL lo convierte a UTC para almacenarlo.
- **Formato de representación:** Generalmente usa el formato ISO 8601 ('YYYY-MM-DD HH:MM:SS'), encerrado en comillas simples.

1.3.1. Ejemplos de casos de uso:

- date: Fechas de nacimiento o fechas de contratacion. (Ej. '1995-08-15').
- time: Horarios de apertura de una tienda. (Ej: '08:20:00').
- timestamptz: Cuándo se realizó exactamente una compra en línea o un log de sistema.(Ej: '2023-11-25 14:35:10-06'). Es vital usarlo con zona horaria para sistemas globales.

1.4. Byte / Bytea (Datos Binarios)

El tipo bytea (Byte Array) se usa para almacenar cadenas binarias çrudas", es decir, datos que no son texto legible (como imágenes pequeñas o archivos compilados).

- **Capacidad de almacenamiento:** Variable, con un límite máximo de 1 GB por campo.
- **Formato de representación:** Se inserta usando un formato "Hex"(hexadecimal) comenzando con .

1.4.1. Ejemplos de casos de uso:

- Guardar la huella digital cifrada o un hash de contraseña (bcrypt). (Ej: ").
- Almacenar avatares o iconos pequeños directamente en la fila del usuario. No se recomienda para archivos gigantes.

1.5. JSON y JSONB

PostgreSQL es famoso por su excelente soporte para datos NoSQL dentro de una base de datos relacional mediante JSON.

- **Capacidad de almacenamiento:** Variable, máximo 1 GB.
  - json: Guarda una copia exacta del texto introducido (incluyendo espacios en blanco y claves duplicadas). Es más rápido para insertar.
  - jsonb (JSON Binary):Descompone el JSON en un formato binario interno. Quita los espacios, ordena las claves y es mucho más rápido de consultar y buscar, además de soportar índices.
- **Formato de representación:** Estructura estándar de JSON en texto.

1.5.1. Ejemplos de casos de uso:

- json: Guardar un registro histórico (log) exacto de una respuesta de una API externa externa que solo necesitas auditar pero no buscar por dentro. (Ej: '"status": 200, "data": "...

  ').
- jsonb: Preferencias de un usuario o configuraciones dinámicas. (Ej: '"tema": .ºscuro", "notificaciones": true, ïdioma": .es

  '). Al ser jsonb, puedes hacer queries para encontrar a todos los usuarios con el tema .ºscuro-ápidamente.

1.6. BLOB (Objetos Grandes / Large Objects)

A diferencia de bytea, la facilidad de Objetos Grandes (Large Objects) en PostgreSQL está diseñada para archivos verdaderamente masivos.

- **Capacidad de almacenamiento:** Hasta 4 TB por archivo.
- **Formato de representación:** Los datos reales se guardan en una tabla oculta del sistema (pg_largeobject). En la tabla de tu aplicación, solo guardas un OID (Object Identifier), que es un número de 4 bytes que funciona como un "puntero.º "link.ªl archivo real.

1.6.1. Ejemplos de casos de uso:

- Almacenar videos en alta definición de 2 GB. En tu tabla guardas un ID, por ejemplo 14563, y PostgreSQL usa funciones especiales (lo_import, lo_export) para leer el video en bloques (chunks) para no saturar la memoria RAM.
- Almacenar documentos PDF enormes, respaldos de disco o pistas de audio pesadas.

2. Referencias:

1. PostgreSQL Tutorial. (s.f.). PostgreSQL Data Types. Recuperado de: https://www.postgresqltutorial.com/postgresql-tutorial/postgresql-data-types/
2. Kerstiens, C. (2015). Postgres Guide to JSON. Craig Kerstiens Blog. Recuperado de: https://www.craigkerstiens.com/2015/05/08/postgres-json-guide/
3. Stack Overflow Community. (s.f.). PostgreSQL: bytea vs Large Objects. Discusiones y convenciones de la comunidad técnica.